IRWA Final Project — Part 2

Indexing and Evaluation | REVIEW DRAFT — not a final submission

Team members / student IDs: [Enter the three actual members and their IDs]

GitHub repository / Part 2 tag: [Insert group repository URL and confirmed tag]

1. Objective and starting point

This part extends the team's Part 1 product preprocessing into an information-retrieval system for a fashion e-commerce collection. We retained product IDs and seven separately processed textual fields. The system retrieves products satisfying every query term and ranks them with TF-IDF. It is then evaluated using the relevance judgments supplied in the course and, once completed, human judgments for five queries.

2. Index construction and preprocessing

The document collection contains 28,080 products. The searchable fields are title, description, brand, category, sub_category, seller and the values of product_details. The preprocessing steps match Part 1: lowercase conversion, a regular-expression tokenizer, removal of NLTK English stop words except "no" and "not", and Porter stemming. Each term is indexed by product ID (pid), field, and within-field positions. Repeated term occurrences are retained for term-frequency computation, while document frequency counts each product at most once per term.

The inverted index supports conjunctive (AND) retrieval by intersecting the product-ID sets for every distinct processed query term. Terms may match in different fields. A query with no useful tokens, or a term absent from the vocabulary, returns no products.

3. Ranking decisions

For each term t, IDF(t) = ln(N/DF(t)) with N = 28,080. The team's original approach applies L2 normalization to raw term-count vectors of documents and of queries, multiplies normalized term frequency by IDF, and uses the dot product of their TF-IDF weights. Thus IDF is multiplied once for the query and once for the document. The implementation is not full cosine similarity because the final weighted vectors are not re-normalized after IDF. Documents are ordered by descending score, with PID tie-breaking.

4. Five group-selected queries

#	Query	AND matches
1	Women pyjamas	193
2	Blue navy jeans for women	43
3	Men dotted tie	3
4	Women silk dress	32
5	Men brown pant	56

These are the five refined queries present in the supplied team Part 2 indexing notebook. The count of matching documents is computed from the full corpus. The queries range from broad searches to more restrictive combinations; their differences help reveal the trade-off introduced by conjunctive search.

Instructor validation — numeric results

5. Predefined queries | K = 10

Query	P@10	R@10	AP@10	F1@10	NDCG@10
1	1.000	0.769	1.000	0.870	1.000
2	0.600	0.600	0.549	0.600	0.706

MAP@10	MRR@10
0.775	1.000

Evaluation methodology and interpretation

6. Validation protocol and assumptions

The provided validation file contains 20 binary labels for each of two predefined queries: “women full sleeve sweatshirt cotton” (Query 1) and “men slim jeans blue” (Query 2). The system ranks the entire product collection first, then restricts the ranking to products with instructor-provided relevance labels. This is the condensed judged-only evaluation protocol: unjudged retrieved products are not automatically classified as irrelevant. Any judged relevant items excluded by Boolean AND remain in the known relevant total for recall, AP and ideal DCG normalization. Missing slots in a top-K result list contribute zero to Precision@K. The chosen cutoff, K = 10, is an explicit assumption because no cutoff is specified in the assignment.

7. What each metric measures

Precision@K: relevant products divided by the first K positions. Recall@K: relevant products retrieved by K divided by all known relevant products. AP@K: averages precision values at ranks where relevant products appear; the denominator here is min(K, the number of known relevant products). F1@K: harmonic mean of P@K and R@K. MAP@K: mean of AP@K across queries. MRR@K: mean of reciprocals of the first relevant hit ranks. NDCG@K: discounted gains for ranked relevant items divided by the ideal attainable gains (binary relevance).

8. Preliminary observed behavior

The resulting metrics show that the engine ranks the judged matching items particularly effectively for Query 1, whereas its performance on Query 2 is less consistent. The two queries have different known-relevance coverage under strict AND retrieval, and reported values are sensitive to the incomplete judgment pool. A high reciprocal-rank score need not imply high recall: it only measures the location of the first relevant result. The final team discussion should inspect sample product titles and verify that these interpretations are supported by the labels and the search results.

9. Main limitations and proposed improvements

Strict AND: longer queries or synonyms may produce few or zero results; test partial-match backoff or query expansion in a subsequent part. Across-field matching: a term may appear only in an irrelevant seller or long description; introduce field weighting and restrict particular attribute matches. Noisy records: repetitive descriptions and inconsistent titles may distort scores; improve data cleaning, deduplication, and attribute extraction. TF-IDF-only ranking: it ignores price, rating and stock, as well as semantic similarity; investigate structured filters, BM25 or Word2Vec in later project stages. Incomplete relevance judgments: the two validation queries only contain judged subsets; report the limited evaluation protocol and explore broader annotation.

Human evaluation and submission completion

10. Five custom queries — human assessment REQUIRED

The assignment asks the group members to serve as expert judges, assigning binary relevance to each retrieved product for the five custom queries. We have generated a full judgment sheet containing 327 unlabeled product-query results, including identifiers, ranking positions, titles, descriptions, metadata and scores. Human labels have not been invented or automatically supplied. Every row must be reviewed: label 1 for a product satisfying the shopper’s actual need, otherwise 0.

Pending: complete `data/human_judgments_ALL_BLANK.csv`; run `python -m project_progress.part_2.evaluate_human --file data/human_judgments_ALL_BLANK.csv`; insert `results/human_metrics.csv` and `results/human_aggregate.csv` here, and write your own specific interpretation comparing the five queries. Custom-query recall measured only within a retrieved judged pool cannot establish recall across the full corpus.

11. Reproducibility

From the repository root, install packages from `requirements.txt`, download NLTK English stopwords once, and run `python -m project_progress.part_2.run_part2`. The script regenerates the preprocessed files when missing, builds the inverted index, ranks queries, reports validation metrics to three decimals and exports the human assessment sheets. Automated unit tests are run with `python -m unittest discover -s tests -v`. Technical details, assumptions and module descriptions are in `project_progress/part_2/README.md`.

12. Submission checks and AI-use disclosure

Still to add: real team names and IDs, the group’s GitHub URL and tag, completed human-query scores and analysis, confirmation of any teacher-specific evaluation convention or K cutoff, and the actual submission date. The repository version and the PDF report must be consistent. Upload the final PDF to Aula Global and share/tag the final code as specified by the course.

Suggested disclosure to review and adapt: “ChatGPT assisted with completing the AND-search implementation, coding and testing evaluation functions, improving reproducibility documentation, and creating an initial report structure. The team reviewed the code and analysis, reproduced the calculations, and performed the human relevance assessments.” Only retain statements that truly reflect your group’s work.

Source materials

IRWA Final Project 2026 Summary; IRWA Project Part 1 and Part 2 instructions; IRWA Modeling I, Indexing & Searching, and Evaluation lecture slides; the supplied team `preprocessing.ipynb` and `part2-indexing.ipynb`; course dataset and validation labels.